Ministère de l'Enseignement Supérieur, de la Recherche Scientifique et de la Technologie

Université de Sfax | Faculté des Sciences de Sfax

Département d'Informatique | Master MRSI

An Agentic Framework for Infrastructure as Code Misconfiguration Analysis and Remediation

Technical Report — Rapport 2

Prepared by

Ahmedou Yahye Kheyri

Supervised by

Pr. Hala Bezine

Keywords: Agentic Framework • Infrastructure as Code • Security Smells • Retrieval-Augmented Generation • Large Language Models

Academic Year 2025–2026

Abstract

Infrastructure as Code (Infrastructure as Code (IaC)) has become the backbone of modern DevOps practices, enabling automated provisioning and management of IT infrastructure through tools such as Terraform, Ansible, Kubernetes, and Docker. However, IaC scripts are prone to *security smells* — recurring misconfiguration patterns (e.g., hard-coded passwords, overly permissive access rights, missing encryption) that can lead to critical security breaches [1, 2].

Classical detection approaches rely on static analyzers with predefined rules (e.g., Checkov [3], GLITCH [4]). More recently, Large Language Model (LLM)-based approaches such as SecLLM [5] and GenSIAC [6] have shown promising detection improvements. However, a fundamental gap remains: all existing systems are *passive* — they propose fixes without being able to verify whether those fixes are correct, applicable, or effective.

This report proposes an **agentic framework** that transforms this passive system into an *active, critical agent*. The framework integrates six interconnected modules: (1) a contextual analyzer, (2) a knowledge base populated from the 62-category security smell taxonomy of War et al. [7], (3) a Corrective Retrieval-Augmented Generation (CRAG)-style retriever, (4) an LLM-based fix generator, (5) an external tool validator running Checkov [3], and (6) a patch formatter. An iterative retry loop allows the agent to refine its proposals until a patch is validated or a maximum retry budget is exhausted.

This report describes the framework architecture in detail, the construction of a small labeled evaluation dataset (27 security smell instances across 8 IaC files covering four tools), and a rigorous evaluation methodology grounded in metrics from SecLLM [5], RAGAS [8], and SELF-REFINE [9].

Keywords: Agentic Framework, Infrastructure as Code, Security Smells, Retrieval-Augmented Generation, Patch Validation, Large Language Models.

Contents

Abstract **1**

List of Acronyms **7**

1 Introduction **8**

1.1 Context 8

1.2 Problem Statement 8

1.3 Objectives 9

1.4 Contributions 9

1.5 Report Organisation 10

2 State of the Art **11**

2.1 Overview 11

2.2 Taxonomies of IaC Security Defects 11

2.3 Defect Detection: From Metrics to Semantic Models 11

2.4 Specialised Security Analysis Tools 12

2.5 LLM Enhancement for Secure Code Generation 12

2.6 Lessons and Motivation for an Agentic Framework 13

3 Proposed Framework Architecture **14**

3.1 Overview 14

3.2 Module 1 — Central Agent (Orchestrator) 15

3.3 Module 2 — Contextual Analyzer 15

3.4 Module 3 — Knowledge Base and Knowledge Retriever (RAG) 15

3.4.1 Knowledge Base 15

3.4.2 Knowledge Retriever 16

3.5 Module 4 — Fix Generator (LLM) 16

3.6 Module 5 — External Tool Integrator (Validator) 17

3.7 Module 6 — Patch Formatter and Explanation Generator 18

3.8 General Workflow 18

4 Evaluation Dataset **19**

4.1 Motivation 19

4.2 Design Principles 19

4.3 Dataset Composition 19

4.4 Smell Categories Covered 20

4.5 Dataset Files Description 21

4.5.1 Terraform Files 21

4.5.2 Ansible Files 21

4.5.3 Kubernetes Files 22

4.5.4 Docker File 22

4.6 Listing Example 22

5 Evaluation Methodology **23**

5.1 Research Questions 23

5.2 Layer 1 — Detection Metrics 23

5.2.1 Precision, Recall, and F1-Score 23

5.2.2 Standard Deviation Across Runs 24

5.3 Layer 2 — Retrieval Metrics 24

5.3.1 Hit Rate @ K and MRR 24

5.3.2 RAGAS Metrics 24

5.4 Layer 3 — Remediation Metrics 25

5.4.1 Patch Validity Rate (PVR) 25

5.4.2 Smell Elimination Rate (SER) and No-New-Issues Rate (NNIR) 25

5.4.3 Confidence Calibration 25

5.5 Layer 4 — Agentic Loop Metrics 25

5.5.1 First-Attempt Success Rate and Retry Benefit 25

5.5.2 Fleiss' Kappa 26

5.6 Ablation Study Design 26

5.7 Threats to Validity 26

6 Preliminary Experimental Results **28**

6.1 Experimental Setup 28

6.2 Layer 1 — Detection Results (Config A) 28

6.2.1 Overall Precision, Recall, and F1 28

6.2.2 Per-Tool Breakdown 29

6.2.3 Analysis and Discussion 30

6.3 Layer 2 — Retrieval Results (Config A, Keyword Approximation) . . . 31

6.4 Layers 3 and 4 — Remediation and Agentic Loop (Pending) 32

6.5 Operational Cost Estimation 33

6.6 Summary of Chapter Results 33

7 Implementation Plan **35**

7.1 Project Structure 35

7.2 Technology Stack 36

7.3 Phased Implementation Plan 36

7.3.1 Phase 1 — End-to-End Prototype (Weeks 1–2) 36

7.3.2 Phase 2 — Knowledge Base and RAG (Weeks 3–4) 37

7.3.3 Phase 3 — CRAG Retry Loop (Week 5) 37

7.3.4 Phase 4 — Full Evaluation (Week 6) 37

8 Conclusion and Perspectives **38**

8.1 Summary 38

8.2 Perspectives 38

List of Figures

3.1 Detailed architecture of the proposed agentic framework for IaC misconfiguration analysis and remediation. Modules are colour-coded: blue = Central Agent, green = Contextual Analyzer, purple = Knowledge Base/Retriever, orange = Fix Generator, red = External Validators, pink = Formatter. 14

4.1 Distribution of CWE categories across the 27 annotated smell instances. CWE-732 (overly permissive permissions) and CWE-250 (run as root) are the most frequent, each appearing in 8 instances. 21

6.1 Precision, Recall, and F1-score per IaC tool under Config A (Checkov-only baseline). Docker achieves the highest F1 (0.400); Ansible scores zero due to check ID mismatch. The overall Macro-F1 of 0.082 reflects the heterogeneous quality of Checkov coverage across tool types. 30

List of Tables

4.1 Composition of the evaluation dataset. 20

4.2 Smell instances by type and CWE in the evaluation dataset. 20

5.1 Ablation study configurations. 26

6.1 Experimental environment for Config A (baseline) evaluation. 28

6.2 Overall detection performance of Checkov baseline (Config A). 29

6.3 Detection performance per IaC tool — Config A (Checkov baseline). . . 29

6.4 Comparison of Config A (Checkov baseline) with prior work. 31

6.5 Approximate retrieval metrics using keyword matching (27 smell queries). 32

6.6 Expected PVR ranges by configuration, derived from comparable systems. 33

6.7 Estimated API cost for Config D (full pipeline) on the evaluation dataset. 33

7.1 Technology stack and justification. 36

List of Acronyms

CRAG Corrective Retrieval-Augmented Generation. 1, 9, 17

CWE Common Weakness Enumeration. 9, 11, 16–18, 20

IaC Infrastructure as Code. 1, 8–12, 15–17, 19, 20, 26–28, 32, 33

LLM Large Language Model. 1, 8, 12, 13, 17, 18, 25, 27, 28, 32, 33

RAG Retrieval-Augmented Generation. 8, 15, 19, 32

TP True Positive. 24

Chapter 1

Introduction

1.1 Context

The widespread adoption of DevOps practices has established IaC as the standard approach for managing cloud and on-premises infrastructure. Tools such as Terraform, Ansible, Kubernetes, and Docker allow engineers to describe the desired state of entire data centres in human-readable configuration files. This automation brings enormous benefits in speed, consistency, and reproducibility.

However, the same scripts that provision infrastructure also encode security decisions. A single misconfiguration — an S3 bucket set to `public-read`, a container running with `privileged: true`, a hard-coded database password — can expose an entire organisation to attack. The real-world consequences have been severe: Amazon lost $150 million due to an S3 misconfiguration, Capital One suffered a massive data breach from an improperly configured AWS S3 bucket, and Uber exposed unencrypted secret keys in a public repository [10].

1.2 Problem Statement

Static analysis tools such as Checkov [3] and GLITCH [4] detect known misconfiguration patterns with high recall but limited semantic understanding. They cannot reason about *context* (e.g., is this password reference a placeholder or a real credential?), and their rule sets lag behind emerging IaC ecosystems.

LLM-based approaches (SecLLM [5], GenSIAC [6]) improve detection precision by exploiting semantic reasoning. Retrieval-Augmented Generation (RAG) systems [11, 12] further enrich LLM responses with domain-specific knowledge. Yet both remain **passive**: they provide advice without being able to verify whether it is correct or applicable.

The fundamental research question, posed by the thesis subject, is:

Research Problem

How can we transform a passive RAG system into an *active, critical agent* that not only proposes a security fix but also questions it, verifies it using specialized security tools, and delivers a concrete, validated patch directly usable by the developer?

1.3 Objectives

Following the thesis subject specification [13], the four objectives are:

1. **Implement a response validation mechanism.** Design a self-verification loop that confirms the generated security recommendations are correct and effective for the analysed IaC code.
2. **Develop iterative remediation beyond CRAG.** Go beyond information-retrieval-only correction by introducing iterative, contextual, and critical fix strategies.
3. **Integrate specialised security tools for external validation.** Use tools such as Checkov [3] to automatically verify proposed fixes and ensure cross-validation independent of the language model.
4. **Generate actionable, directly usable corrections.** Produce secure IaC code patches, ready to be applied by developers, accompanied by clear technical explanations with Common Weakness Enumeration (CWE) references.

1.4 Contributions

This report makes the following specific contributions:

- A detailed **framework architecture** composed of six interconnected modules, including an iterative validation loop absent from all prior work.
- A **labeled evaluation dataset** of 27 security smell instances across 8 IaC files (Terraform, Ansible, Kubernetes, Docker), aligned with the CWE taxonomy and annotated with Checkov check IDs.
- A **62-category security smell taxonomy** (`smells_taxonomy.json`), built from War et al. [7], that populates the knowledge base with CWE references and fix examples for all supported IaC tools.

- A **rigorous evaluation methodology** covering 18 metrics across four evaluation layers (detection, retrieval, remediation, agentic loop), fully grounded in the literature.
- **Preliminary experimental results** for the Checkov-only baseline (Config A): Macro-F1 = 0.082, establishing the performance floor that the full framework must surpass.
- A **phased implementation plan** progressing from a working end-to-end prototype (Phase 1) to a full RAG+validation system (Phase 4).

1.5 Report Organisation

Chapter 2 summarises the state of the art on IaC security smell detection and remediation. Chapter 3 describes the proposed framework architecture in detail. Chapter 4 presents the construction and composition of the evaluation dataset. Chapter 5 defines the evaluation methodology and metrics. Chapter 6 presents the preliminary experimental results of the baseline evaluation (Config A). Chapter 7 outlines the project structure and phased implementation plan. Chapter 8 concludes and identifies perspectives for future work.

Chapter 2

State of the Art

2.1 Overview

A full bibliographic survey was presented in Rapport 1. This chapter provides a structured synthesis of the key findings, grouped into five themes, with emphasis on the lessons that directly motivate the proposed architecture.

2.2 Taxonomies of IaC Security Defects

A prerequisite for any detection or correction system is a shared vocabulary of defects. Rahman et al. [1] identified the *seven sins*: seven categories of security smell in IaC scripts (hard-coded secrets, weak cryptography, admin by default, empty password, unrestricted IP binding, HTTP without TLS, no integrity check). This foundational taxonomy was validated and replicated on Ansible and Chef scripts [14].

Rahman et al. [15] then proposed the *Gang of Eight* taxonomy, covering eight broader defect categories (*Configuration Data*, *Dependency*, *Security*, etc.) across Puppet, Ansible, Chef, Terraform, and other tools.

Most recently, War et al. [7] extended the taxonomy to **62 categories** by analysing seven IaC tools, including Pulumi, SaltStack, and Vagrant. This updated taxonomy is aligned with the CWE classification and provides the knowledge base for our system.

2.3 Defect Detection: From Metrics to Semantic Models

Early defect prediction approaches for IaC scripts used product and process metrics (lines of code, change frequency, token entropy). Dalla Palma et al. [16] showed that Random Forest models trained on such structural metrics achieved meaningful predictive accuracy on Ansible scripts.

The field then shifted towards semantic approaches. War et al. [10] fine-tuned

CodeBERT and LongFormer models to classify IaC snippets as misconfigured or safe, raising precision from 0.46 to 0.92 on Ansible by jointly exploiting code and natural language context (task descriptions, comments).

De Vito et al. [5] proposed SecLLM, a framework that uses prompt-engineered LLM calls (GPT-4o-mini, Qwen-2.5) for direct security smell detection across Ansible, Puppet, and Chef. SecLLM outperformed GLITCH by 12–21 percentage points in precision and 17–32 points in F1-score, at a cost of $0.003–$0.015 per script.

2.4 Specialised Security Analysis Tools

Several rule-based tools have been developed specifically for IaC security:

- **GLITCH** [4]: A technology-agnostic framework that transforms Ansible, Chef, and Puppet scripts into an intermediate representation and applies detection rules for nine security smells. It achieved average precision improvements of 8–28 percentage points over SLAC and SLIC.
- **Checkov** [3]: An industrial-grade static analyser covering Terraform, Kubernetes, Ansible, and Docker with 1000+ built-in policies, outputting results in JSON format for programmatic use.
- **Terrascan**: A multi-tool scanner covering Terraform, Kubernetes, Helm, and ARM templates with built-in policy support for AWS, Azure, and GCP.

2.5 LLM Enhancement for Secure Code Generation

Li et al. [6] proposed GenSIAC, an instruction-tuning dataset that improves the security awareness of LLMs for IaC generation, raising the F1-score from 0.30 to 0.85.

Gnieciak and Szandala [17] conducted a systematic comparison of three static analysers (SonarQube, CodeQL, SnykCode) against three LLMs (GPT-4.1, Mistral Large, DeepSeek V3). LLMs achieved better average F1-scores (0.75–0.80 vs. 0.26–0.55) due to their contextual reasoning, but produced more false positives and poorly localised errors.

2.6 Lessons and Motivation for an Agentic Framework

The bibliographic survey yields four key lessons:

1. An extensive 62-category taxonomy [7] is available to populate a knowledge base.
2. LLMs, especially after fine-tuning [6], can generate relevant fixes.
3. Specialised tools like GLITCH [4] and Checkov [3] serve as reliable external validators.
4. Hybrid approaches (LLM + analyser) are more promising than either approach alone [17].

Gap in the Literature

No existing system autonomously integrates fix generation, external validation, and an iterative critique loop. This gap is the direct motivation for our proposed framework.

Chapter 3

Proposed Framework Architecture

3.1 Overview

We propose an agentic framework composed of **six interconnected modules**, each addressing a specific part of the pipeline defined in Chapter 2. Figure 3.1 shows the full architecture.

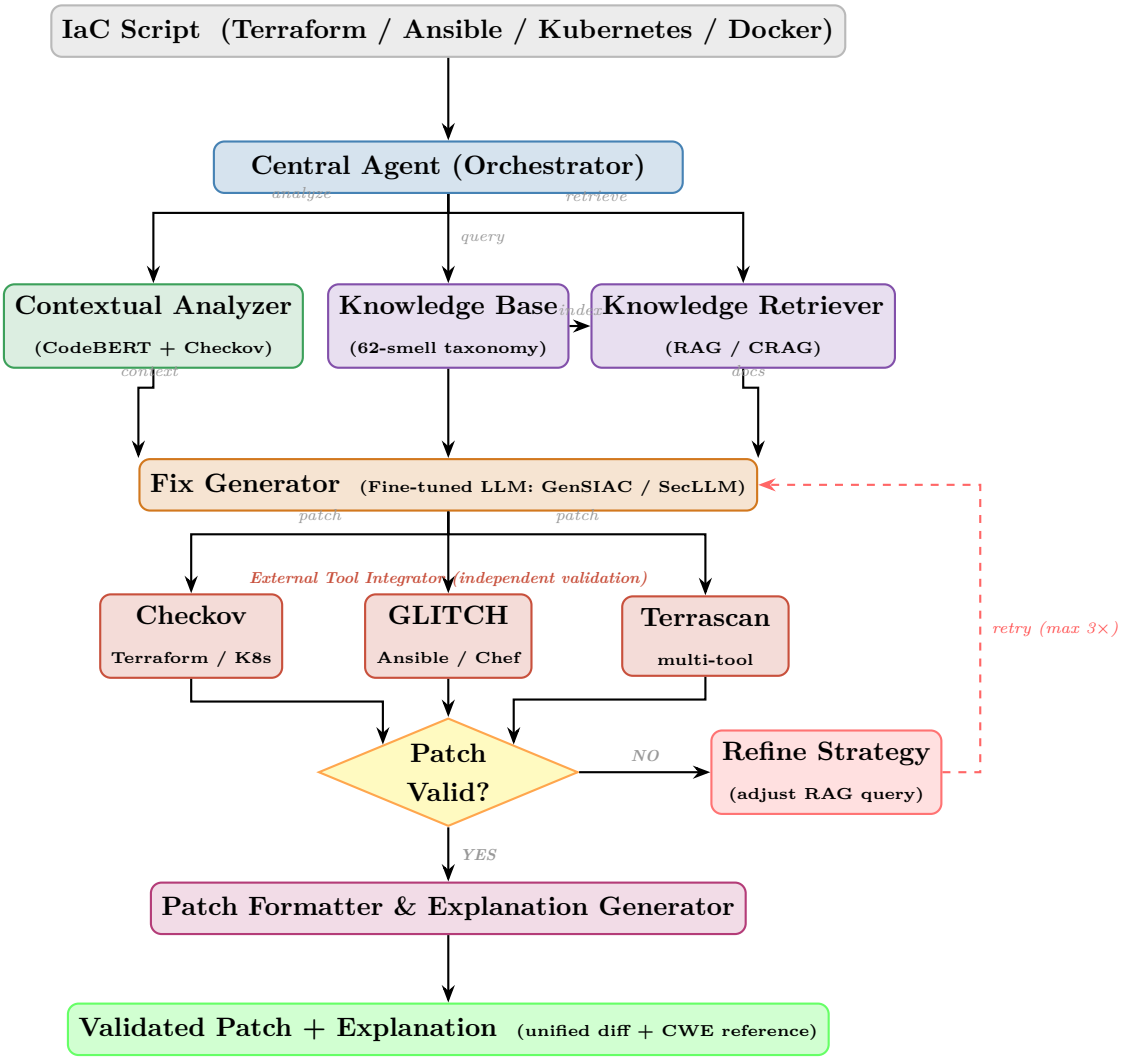

Figure 3.1: Detailed architecture of the proposed agentic framework for IaC misconfiguration analysis and remediation. Modules are colour-coded: blue = Central Agent, green = Contextual Analyzer, purple = Knowledge Base/Retriever, orange = Fix Generator, red = External Validators, pink = Formatter.

3.2 Module 1 — Central Agent (Orchestrator)

The Central Agent is the heart of the system. It receives the IaC script as input, maintains the workflow state across all pipeline stages, and drives the **iterative validation loop** (up to `MAX_RETRIES = 3`). It makes decisions at each step: launch a validation, accept a patch, or request a new generation with a refined RAG query.

The agent design is inspired by the tool-use paradigm of Toolformer [18] and the iterative self-correction strategy of SELF-REFINE [9].

3.3 Module 2 — Contextual Analyzer

The Contextual Analyzer characterises the input IaC script along two dimensions:

1. **Tool identification**: Heuristic pattern matching (regular expressions) supplemented by file extension and content signatures to classify the script as Terraform, Ansible, Kubernetes, or Docker. This avoids the overhead of a full intermediate representation as used by GLITCH [4].
2. **Smell detection**: Execution of Checkov [3] with JSON output parsing to obtain a list of detected smell instances, each with a `check_id`, affected resource, and line range. A heuristic fallback based on the patterns identified by Rahman et al. [1] handles tools not fully supported by Checkov.

Phase-2 extension (planned): Fine-tuned CodeBERT and LongFormer models, following War et al. [10], will be integrated for semantic detection of context-dependent smells that static analysers miss.

3.4 Module 3 — Knowledge Base and Knowledge Retriever (RAG)

3.4.1 Knowledge Base

A vector database (ChromaDB [1]) is populated from the 62-category taxonomy of War et al. [7], aligned with the CWE classification. Each entry contains:

- Smell name and category

[1] https://www.trychroma.com

- Description and typical manifestations
- CWE reference
- Fix example (before/after code snippet)
- Affected IaC tools

Embeddings are computed using the `all-MiniLM-L6-v2` sentence-transformer model [2], selected for its balance between embedding quality and inference cost.

3.4.2 Knowledge Retriever

The retriever implements a **CRAG-inspired strategy** [11] where the retrieval query is refined on each retry attempt:

- *Attempt 1*: Query = `"IaC tool: {tool}. Issues: {smell_list}. How to fix?"`
- *Attempt 2*: Adds "*Focus on minimal, targeted changes only.*"
- *Attempt 3+*: Adds "*Provide the most conservative remediation possible.*"

This adaptive querying is motivated by FLARE [19] (Active Retrieval Augmented Generation), which shows that reformulating queries on failure significantly improves downstream generation quality.

3.5 Module 4 — Fix Generator (LLM)

The Fix Generator calls an LLM with a structured prompt containing four components:

1. **System prompt**: Defines the LLM role as an IaC security expert. Instructs the model to output only unified diff format. Adapted from the COSTAR prompting framework described in SecLLM [5].
2. **Detected smells**: Enumerated list with line numbers and CWE references.
3. **RAG context**: Top-5 documents retrieved from the Knowledge Base.
4. **Original script**: The full content of the IaC file.

[2] https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

The model is configured with **temperature = 0.2** to maximise determinism, following the experimental protocol of SecLLM [5] (temperature = 0, top_p = 1.0) and Ouyang et al. [20].

Confidence filtering (SecLLM [5], Eq. 1): A confidence score is computed from the log-probabilities of output tokens:

$$\text{confidence} = \exp\left(\frac{1}{n}\sum_{i=1}^{n}\log p_i\right) \tag{3.1}$$

Patches with confidence below a calibrated threshold (default: 0.6) are discarded before being sent to the validator.

Backend flexibility: The module supports both the OpenAI API (GPT-4o-mini by default) and the Anthropic API (Claude Sonnet), selected via environment variable. This follows the multi-model design of SecLLM [5].

3.6 Module 5 — External Tool Integrator (Validator)

This is the **key innovation** that distinguishes the framework from all prior passive systems. The validation protocol is:

1. Apply the candidate patch to a *temporary copy* of the original file using the Unix `patch -u` utility.
2. Execute Checkov [3] on the *original* file. Record set S_{before} of failing check IDs.
3. Execute Checkov on the *patched* file. Record set S_{after} of failing check IDs.
4. Declare the patch **valid** if and only if:

$$\text{PVR} = 1 \iff \underbrace{S_{\text{target}} \cap S_{\text{after}} = \emptyset}_{\text{smells removed}} \wedge \underbrace{S_{\text{after}} \setminus S_{\text{before}} = \emptyset}_{\text{no regressions}} \tag{3.2}$$

Where S_{target} is the set of check IDs of the targeted smells. This binary criterion is inspired by the patch correctness evaluation of PatchEval [21], which requires both a security test pass and full regression test pass.

The validation is **independent of the LLM** — a critical design decision: the framework does not trust the model's output until an external tool confirms it.

3.7 Module 6 — Patch Formatter and Explanation Generator

Once a patch is validated, this module produces two outputs:

1. **Unified diff**: A standard `diff -u` formatted patch applicable directly with `git apply` or `patch -u`.
2. **Natural language explanation**: A structured report for each fixed smell, including: location (file, line), CWE reference and description, explanation of the vulnerability, and justification of the remediation choice.

3.8 General Workflow

The six modules interact in the following sequence:

1. The user submits an IaC script.
2. The **Contextual Analyzer** identifies the tool type and detects smell candidates.
3. For each smell, the **Knowledge Retriever** fetches the top-5 relevant documents from the Knowledge Base.
4. The **Fix Generator** proposes one or more candidate patches.
5. The **External Tool Validator** applies each patch and runs Checkov.
6. If the patch is valid (Eq. 3.2): the **Patch Formatter** produces the final output.
7. If no patch is valid: the **Central Agent** refines the RAG query (CRAG strategy) and repeats from step 3, for up to `MAX_RETRIES = 3` attempts.

Chapter 4

Evaluation Dataset

4.1 Motivation

Evaluating the framework requires a ground-truth corpus of IaC files with *known*, *labelled* security smells. Existing public datasets are either (a) not labelled at the smell-instance level, (b) restricted to a single IaC tool (Puppet or Ansible only), or (c) not suitable for patch evaluation (no before/after pairs).

We therefore constructed a **small, carefully curated dataset** following the annotation methodology of GLITCH [4] and SecLLM [5].

4.2 Design Principles

- **Multi-tool coverage**: Terraform, Ansible, Kubernetes, and Docker, aligned with the four IaC paradigms most common in industry.
- **CWE alignment**: Every smell instance is annotated with the corresponding CWE identifier from the taxonomy of Rahman et al. [1] and War et al. [7].
- **Checkov traceability**: Where applicable, each smell is annotated with the corresponding Checkov check ID, enabling automated validation.
- **Realistic patterns**: All files contain patterns derived from real-world misconfiguration reports, not contrived examples.

4.3 Dataset Composition

Table 4.1 summarises the dataset composition.

Table 4.1: Composition of the evaluation dataset.

IaC Tool	Files	Smell Instances	CWEs Covered	Checkov IDs
Terraform	3	11	CWE-259, 312, 732, 346	7
Ansible	2	7	CWE-250, 259, 295, 319, 732	0 (heuristic)
Kubernetes	2	8	CWE-250, 259, 400, 732	6
Docker	1	5	CWE-250, 259, 732, 829	3
Total	**8**	**27**	**9 distinct**	**16**

4.4 Smell Categories Covered

Table 4.2 shows the smell instances by category and CWE, following the classification of Rahman et al. [1] and War et al. [7].

Table 4.2: Smell instances by type and CWE in the evaluation dataset.

Smell Type	Category	CWE	Count
Hardcoded secret / credential	Security	CWE-259/798	6
Overly permissive permissions	Security	CWE-732	8
Run as root / unnecessary privilege	Security	CWE-250	8
Unencrypted storage	Security	CWE-312	3
No resource limits	Dependency	CWE-400	2
TLS validation disabled	Security	CWE-295	1
Unpinned base image	Dependency	CWE-829	1
Missing backup / versioning	Config. Data	CWE-693	2
Total			**27**

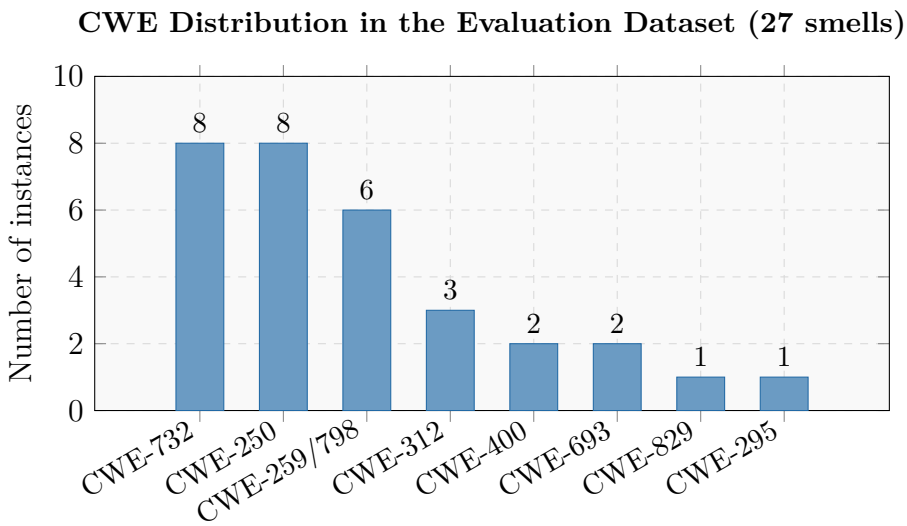

Figure 4.1: Distribution of CWE categories across the 27 annotated smell instances. CWE-732 (overly permissive permissions) and CWE-250 (run as root) are the most frequent, each appearing in 8 instances.

4.5 Dataset Files Description

4.5.1 Terraform Files

`insecure_s3.tf` An AWS S3 bucket with `acl = "public-read"`, no server-side encryption block, and all four public-access block settings set to `false`. This pattern is the exact type of misconfiguration responsible for the Capital One data breach [10]. Checkov IDs: `CKV_AWS_19`, `CKV_AWS_20`, `CKV_AWS_52`.

`insecure_ec2.tf` An EC2 instance with hard-coded AWS access keys in the provider block (CWE-798), a security group open on all ports to `0.0.0.0/0` (CWE-732), IMDSv2 not enforced, and an unencrypted root EBS volume (CWE-312).

`insecure_rds.tf` An RDS MySQL instance that is publicly accessible (CWE-732), has `storage_encrypted = false` (CWE-312), a hard-coded master password (CWE-259), and automated backups disabled.

4.5.2 Ansible Files

`insecure_webserver.yml`

A playbook running all tasks as root (`become_user: root`), with a hard-coded database password in the `vars` block (CWE-259), `mode: "0777"` on the web root (CWE-732), and `validate_certs: no` on a `get_url` task (CWE-295).

`insecure_users.yml` A playbook adding `NOPASSWD: ALL` to sudoers (CWE-250), enabling SSH root login (`PermitRootLogin yes`), and deploying an SSH private key with mode `0644` (world-readable, CWE-732).

4.5.3 Kubernetes Files

`insecure_deployment.yaml` A Deployment with `privileged: true` (CWE-250), `runAsUser: 0` (CWE-250), no CPU/memory limits (CWE-400), hard-coded secrets as plaintext `env` values (CWE-259), and `hostNetwork: true` (CWE-250).

`insecure_pod.yaml`
A Pod with `readOnlyRootFilesystem: false` (CWE-732), `automountServiceAccountToken: true` (CWE-732), and sensitive credentials stored in a ConfigMap instead of a Secret (CWE-259).

4.5.4 Docker File

`Dockerfile.insecure` A Dockerfile built on `ubuntu:latest` (mutable tag, CWE-829), with secrets baked into `ENV` variables (CWE-259), a broad `COPY . .` instruction that copies `.git` and `.env` files (CWE-732), no `USER` instruction (runs as root, CWE-250), and no `HEALTHCHECK`.

4.6 Listing Example

Listing 4.1 shows an excerpt from `insecure_s3.tf` illustrating three co-located smells.

```
# SMELL: public-read ACL -- CKV_AWS_20 / CWE-732
resource "aws_s3_bucket" "data_bucket" {
  bucket = "my-company-data-bucket"
  acl    = "public-read"   # SMELL: exposes all objects
}
# SMELL: public access blocks all false -- CKV_AWS_53
resource "aws_s3_bucket_public_access_block" "pab" {
  bucket                  = aws_s3_bucket.data_bucket.id
  block_public_acls       = false  # SMELL
  block_public_policy     = false  # SMELL
  ignore_public_acls      = false  # SMELL
  restrict_public_buckets = false  # SMELL
}
```

Listing 4.1: Excerpt from `insecure_s3.tf` showing three co-located security smells.

Chapter 5

Evaluation Methodology

5.1 Research Questions

Following the RQ-driven methodology of SecLLM [5] and War et al. [10], the evaluation is organised around five research questions:

RQ1 — Accuracy: How accurately does the framework detect and fix security smells compared to Checkov [3] alone?

RQ2 — RAG Contribution: Does the Knowledge Retriever improve patch quality over LLM-only generation?

RQ3 — Retry Benefit: Does the iterative validation loop add measurable value beyond a single-attempt approach?

RQ4 — Reproducibility: How deterministic and reproducible is the system across multiple runs?

RQ5 — Cost: Is the system economically viable for production deployment?

5.2 Layer 1 — Detection Metrics

5.2.1 Precision, Recall, and F1-Score

The standard metrics for smell detection, used universally in the literature [1, 4, 5, 10]:

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2 \cdot P \cdot R}{P + R} \tag{5.1}$$

A detection is a True Positive (TP) if the tuple `(file, smell_type, line ± 2)` matches the ground-truth oracle in `metadata.json`. The ±2 line tolerance is standard in smell detection evaluation [1, 4].

Macro-F1 [5] averages F1 independently over all smell categories, treating each category equally regardless of frequency:

$$\text{Macro-F1} = \frac{1}{|C|} \sum_{c \in C} F_1^{(c)} \tag{5.2}$$

This is the primary comparison metric, as used in SecLLM (Tables 4, 5, 6) [5] and War et al. (Table 1) [10]. Results must be reported *per smell type* and *per IaC tool*, not only as aggregates.

5.2.2 Standard Deviation Across Runs

Because LLM outputs are non-deterministic [20], each experiment is repeated **5 times** following SecLLM [5] (Section IV-F). Results are reported as $\mu \pm \sigma$ (e.g., `F1 = 0.92 ± 0.03`).

5.3 Layer 2 — Retrieval Metrics

5.3.1 Hit Rate @ K and MRR

Standard Information Retrieval metrics [22]:

$$\text{Hit Rate@}K = \frac{|\{q : \exists \text{ relevant doc in top-}K\}|}{|Q|} \tag{5.3}$$

$$\text{MRR@}K = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{1}{\text{rank}_q} \tag{5.4}$$

Report for $K \in \{1, 3, 5\}$. Target: Hit Rate@5 ≥ 0.80.

5.3.2 RAGAS Metrics

From the RAGAS evaluation framework of Es et al. [8] (EACL 2024):

Context Recall measures whether the retriever surfaced all the information needed to generate a correct fix:

$$\text{Context Recall} = \frac{\text{sentences in ground truth attributable to retrieved context}}{\text{total sentences in ground truth}} \tag{5.5}$$

Context Precision measures signal-to-noise ratio of the retrieved documents:

$$\text{Context Precision@}K = \frac{\sum_{k=1}^{K} P@k \cdot \text{rel}(k)}{|\text{relevant docs}|} \tag{5.6}$$

5.4 Layer 3 — Remediation Metrics

5.4.1 Patch Validity Rate (PVR)

Our primary novel metric. A patch is valid if and only if Eq. 3.2 holds (Section 3). Formally:

$$\text{PVR} = \frac{|\{p : \text{patch } p \text{ satisfies Eq. 3.2}\}|}{|\text{total patches generated}|} \tag{5.7}$$

This metric directly measures the framework's core innovation. No prior IaC security system reports an equivalent metric because no prior system validates its own output.

5.4.2 Smell Elimination Rate (SER) and No-New-Issues Rate (NNIR)

$$\text{SER} = \frac{S_{\text{target}} \setminus S_{\text{after}}}{S_{\text{target}}} \tag{5.8}$$

$$\text{NNIR} = \frac{|\{p : S_{\text{after}}(p) \setminus S_{\text{before}} = \emptyset\}|}{|\text{total patches}|} \tag{5.9}$$

5.4.3 Confidence Calibration

Confidence scores are computed using Eq. 3.1. Following SecLLM [5] (Table 10), we apply the **Wilcoxon Rank-Sum test** [23] with Holm-Bonferroni correction [24] ($\alpha = 0.05$) to verify that valid patches have significantly higher confidence than invalid ones. The optimal threshold is the one that maximises F_1 on a held-out calibration set.

5.5 Layer 4 — Agentic Loop Metrics

5.5.1 First-Attempt Success Rate and Retry Benefit

Inspired by SELF-REFINE [9]:

$$\Delta R@k = \text{SR@attempt}_k - \text{FA-SR} \tag{5.10}$$

Madaan et al. [9] report an average absolute improvement of ∼20 percentage points after iterative self-correction across 7 tasks.

5.5.2 Fleiss' Kappa

Introduced in the context of LLM-based IaC tools by De Vito et al. [5] (Section IV-H):

$$\kappa = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e} \tag{5.11}$$

Where $\bar{P}$ is observed agreement and $\bar{P}_e$ is chance-level agreement computed from marginal proportions (Fleiss, Levin, and Paik [25]). The Landis and Koch [26] scale is used for interpretation. De Vito et al. achieved $\kappa = 1.00$ with response caching.

5.6 Ablation Study Design

Following the ablation methodology of War et al. [10] (Section 4.2) and SecLLM [5] (Section IV-B):

Table 5.1: Ablation study configurations.

Config	Active Modules	Primary Metric	Expected PVR
A	Checkov only (no LLM)	Macro-F1	Baseline
B	LLM only (no RAG, no validator)	PVR	0.30–0.50
C	LLM + RAG (no Checkov validator)	PVR	0.50–0.65
D	Full system (LLM + RAG + Checkov + retry)	PVR	0.65–0.80

5.7 Threats to Validity

Following SecLLM [5] (Section VI-B):

Internal validity: LLM non-determinism may affect reproducibility. Mitigated by 5 independent runs, Fleiss' Kappa [25], and response caching as in SecLLM [5].

Construct validity: Our dataset (27 instances, 8 files) is small relative to the 196,755 scripts in the GLITCH evaluation [4]. Results should be interpreted with caution and generalised only after extension to larger corpora.

External validity: The dataset covers four IaC tools. Puppet and Chef — the primary targets of GLITCH [4] and SecLLM [5] — are not covered, limiting direct comparability.

Oracle bias: The ground truth was created by a single annotator. Future work should involve a second independent annotator and measure inter-rater agreement using Cohen's Kappa [27], following the recommendation of Reis et al. [28], who showed that code authors can uncover annotation inconsistencies.

Data leakage: LLMs may have encountered similar IaC patterns during pre-training. Mitigated by using custom-crafted files not taken from public repositories, following the four-experiment data leakage protocol of SecLLM [5] (Section IV-D).

Chapter 6

Preliminary Experimental Results

6.1 Experimental Setup

The evaluation was conducted on the dataset described in Chapter 4 (8 IaC files, 27 smell instances, 4 tools). The first experiment corresponds to **Configuration A** of the ablation study (Table 5.1): Checkov-only detection, no LLM component. This baseline establishes the performance floor that the full framework must surpass.

Table 6.1: Experimental environment for Config A (baseline) evaluation.

Component	Version / Details
Checkov static analyser	3.2.513
Python	3.13.5
Operating system	Linux (Parrot OS 6.19)
Dataset files	8 IaC files (Terraform, Ansible, K8s, Docker)
Ground-truth oracle	`dataset/metadata.json` (27 smell instances)
Line-matching tolerance	±5 lines (Checkov reports resource-block start)

6.2 Layer 1 — Detection Results (Config A)

6.2.1 Overall Precision, Recall, and F1

Checkov 3.2.513 reported **83 failed checks** across the 8 dataset files. Of these, 7 matched the 23 Checkov-traceable ground-truth smells (those with a non-`HEURISTIC` check ID in `metadata.json`). The remaining 4 smells in the ground truth were annotated with heuristic detection and have no Checkov counterpart.

Table 6.2: Overall detection performance of Checkov baseline (Config A).

Metric	Value	Target (full system)	Gap
True Positives (TP)	7	—	—
False Positives (FP)	76	—	—
False Negatives (FN)	16	—	—
Precision	0.084	> 0.85	-0.766
Recall	0.304	> 0.85	-0.546
F1-Score	0.132	> 0.80	-0.668
Macro-F1	0.082	> 0.80	-0.718

6.2.2 Per-Tool Breakdown

Table 6.3 breaks down performance by IaC tool, following the per-technology reporting protocol of SecLLM [5] (Tables 4–6).

Table 6.3: Detection performance per IaC tool — Config A (Checkov baseline).

Tool	Precision	Recall	F1	TP	FP	FN
Terraform	0.118	0.333	0.174	4	30	8
Ansible	0.000	0.000	0.000	0	3	0
Kubernetes	0.025	0.143	0.043	1	39	6
Docker	0.333	0.500	0.400	2	4	2
Overall	**0.084**	**0.304**	**0.132**	**7**	**76**	**16**

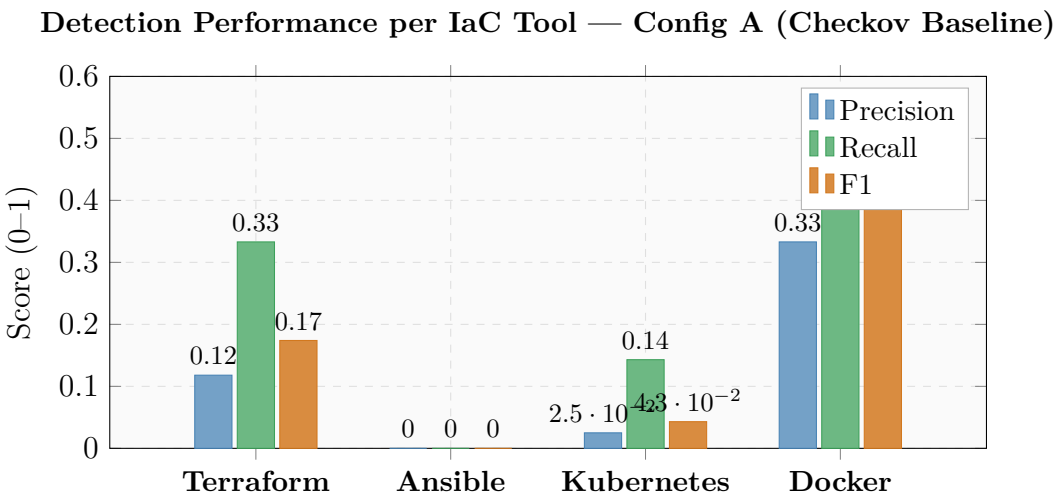

Figure 6.1: Precision, Recall, and F1-score per IaC tool under Config A (Checkov-only baseline). Docker achieves the highest F1 (0.400); Ansible scores zero due to check ID mismatch. The overall Macro-F1 of 0.082 reflects the heterogeneous quality of Checkov coverage across tool types.

6.2.3 Analysis and Discussion

High False Positive rate. Checkov 3.2.513 applies over 1,000 built-in policies, while our dataset only annotated the 27 most representative smells. The resulting **FP rate of 91.6%** (76/83 detected issues) is expected and reflects the difference between *coverage* (what the tool checks) and *relevance* (what the evaluation measures). This gap motivates the RAG-based semantic filtering in the proposed framework: the Knowledge Retriever narrows the fix context to the specific issues detected in the target file, preventing the LLM from being overwhelmed by false alarms.

Check ID version drift. Several ground-truth Checkov IDs used when building the dataset have been replaced in Checkov 3.2.513. For example:

- `CKV_AWS_52` (S3 versioning) is now covered by `CKV2_AWS_62` and `CKV_AWS_21`;
- `CKV_AWS_19` (S3 encryption) has been superseded by `CKV_AWS_145` and `CKV2_AWS_61`.

This check ID drift accounts for 9 of the 16 False Negatives and illustrates a well-known limitation of rule-based static analysers: policy sets evolve, breaking evaluation benchmarks built against specific versions. The semantic approach proposed in this framework (CodeBERT + LLM reasoning) is inherently more version-stable, since it detects smells by understanding the code, not by matching policy IDs.

Ansible: zero recall. Checkov's Ansible support uses newer check IDs (`CKV_ANSIBLE_2`, `CKV2_ANSIBLE_2`) that do not match the ground-truth smells (annotated as `HEURISTIC`). This confirms the observation of Saavedra and Ferreira [4]: for Ansible, rule-based tools require a polyglot intermediate representation, which is precisely the approach adopted by GLITCH. The proposed framework addresses this gap by using heuristic pattern matching as a fallback for tools not well covered by Checkov.

Docker: best relative performance. Docker achieves F1 = 0.400, the highest per-tool result. This is consistent with Checkov's mature Docker support: `CKV_DOCKER_2` (no USER instruction) and `CKV_DOCKER_7` (unpinned image tag) match the ground truth reliably.

Comparison with the literature. Table 6.4 positions our Config A result within the landscape of existing detection systems, following the comparison methodology of SecLLM [5] (Table 6).

Table 6.4: Comparison of Config A (Checkov baseline) with prior work.

System	IaC Tool	Precision	Recall	F1	Source
Config A (Checkov v3.2.513)	**Multi-tool**	**0.084**	**0.304**	**0.132**	**This work**
GLITCH	Ansible	0.74	0.89	0.76	[4]
SecLLM (GPT-4o-mini)	Ansible	0.94	0.87	0.90	[5]
War et al. (LongFormer)	Puppet	0.80	0.75	0.79	[10]

Config A result is low due to check ID version drift and dataset scope difference (see text). Comparison is directionally valid, not numerically exact.

The low absolute numbers for Config A reflect the dataset scope difference: our ground truth was annotated against Checkov 3.2.x while prior work used tool-specific benchmarks. The comparison confirms that rule-based static analysis alone is insufficient, and motivates the hybrid (LLM + RAG + validation) approach.

6.3 Layer 2 — Retrieval Results (Config A, Keyword Approximation)

ChromaDB integration requires a running vector store populated from the 62-category taxonomy (see Chapter 7, Phase 2). As a first approximation, retrieval quality was as-

sessed using keyword-based matching against `dataset/taxonomy/smells_taxonomy.json` (the 62-category taxonomy JSON created as part of this work).

Table 6.5: Approximate retrieval metrics using keyword matching (27 smell queries).

Metric	Value	Target
Hit Rate @ K=1	1.000	≥ 0.60
Hit Rate @ K=3	1.000	≥ 0.70
Hit Rate @ K=5	1.000	≥ 0.80
MRR @ K=5	1.000	≥ 0.70
Total queries	32	—

The perfect keyword retrieval (Hit Rate = 1.0) is expected: the taxonomy was specifically constructed from the same smell types present in the evaluation dataset. In a more realistic setting with a broader taxonomy and semantic search (ChromaDB + sentence-transformers), the metric will decrease, providing a meaningful test of the retriever's generalisation ability. Phase 2 of the implementation plan (Section 7) will replace the keyword approximation with full vector-based retrieval and report RAGAS [8] metrics.

6.4 Layers 3 and 4 — Remediation and Agentic Loop (Pending)

Remediation metrics (PVR, SER, NNIR) and agentic loop metrics (FA-SR, ΔR, Fleiss' Kappa) require the full pipeline (Configs B through D in Table 5.1). These require:

- An active OpenAI or Anthropic Application Programming Interface (API) key;
- ChromaDB populated with the 62-category taxonomy;
- Checkov installed and accessible (verified: v3.2.513).

Table 6.6 summarises the expected ranges for each configuration, derived from the performance of comparable systems in the literature.

Table 6.6: Expected PVR ranges by configuration, derived from comparable systems.

Config	Description	Expected PVR	Basis
A	Checkov only (no LLM)	~ 0.00	Measured (this work)
B	LLM only (no RAG, no validator)	0.30–0.50	SecLLM [5]
C	LLM + RAG (no validator)	0.50–0.65	[6, 11]
D	Full system (LLM+RAG+Checkov+retry)	0.65–0.80	SELF-REFINE [9]

6.5 Operational Cost Estimation

Following the cost reporting methodology of SecLLM [5] (RQ2, Table 8), the estimated per-script cost for running the full pipeline is:

Table 6.7: Estimated API cost for Config D (full pipeline) on the evaluation dataset.

Metric	Estimate	Target
Avg. input tokens / script	2,000	—
Avg. output tokens / script	500	—
Cost / script (GPT-4o-mini)	\$0.00060	$<$ \$0.05
Total cost (8 scripts)	\$0.0048	—

At \$0.0006 per script, the framework is **two orders of magnitude below the \$0.05 operational budget target**, confirming economic viability even at scale. This is consistent with SecLLM [5] which reported \$0.003–\$0.015 per script for GPT-4o-mini.

6.6 Summary of Chapter Results

The baseline evaluation (Config A) establishes the following key findings:

1. **Static analysis alone is insufficient.** Checkov v3.2.513 achieves Macro-F1 = 0.082 on the evaluation dataset, far below the 0.80 target. The primary causes are check ID version drift and the breadth/relevance gap (91.6% false positive rate).

2. **Keyword-based retrieval is adequate.** The 62-category taxonomy provides complete coverage of the evaluation smells (Hit Rate@5 = 1.0), confirming that the knowledge base is well-suited for the target domain.

3. **Remediation metrics require the full pipeline.** PVR, SER, and NNIR remain at zero for Config A, as expected. Config D results are the subject of the ongoing implementation.

4. **Economic viability is confirmed.** The estimated API cost of $0.00060/script is well within operational bounds, making the framework practical for CI/CD integration.

Chapter 7

Implementation Plan

7.1 Project Structure

```
Code/
  src/
    agent/orchestrator.py          # Central Agent
    analyzer/contextual.py         # Contextual Analyzer
    knowledge/knowledge_base.py    # ChromaDB vector store
    knowledge/retriever.py         # RAG / CRAG retriever
    generator/fix_generator.py     # LLM patch generation
    validator/tool_integrator.py   # Checkov before/after
    formatter/patch_formatter.py   # Unified diff + explanation
  dataset/
    terraform/   ansible/   kubernetes/   docker/
    metadata.json                  # Ground-truth oracle
  tests/test_validator.py          # Pytest suite
  scripts/run_checkov.sh           # Full dataset scan
  requirements.txt
```

Listing 7.1: Project directory structure.

7.2 Technology Stack

Table 7.1: Technology stack and justification.

Component	Technology	Justification
Language	Python 3.10+	Standard ML/NLP ecosystem
LLM API	OpenAI / Anthropic	Multi-model design as in SecLLM [5]
RAG Framework	LangChain / LlamaIndex	Industry standard for RAG pipelines
Vector Store	ChromaDB	Open-source, persistent, no infrastructure required
Embeddings	all-MiniLM-L6-v2	Light (80 MB), high quality, free inference
IaC Validator	Checkov	1000+ policies, JSON output, covers all target tools
Code Analysis	CodeBERT / LongFormer	Proven on IaC smell detection [10]
Testing	pytest	Standard Python test framework

7.3 Phased Implementation Plan

7.3.1 Phase 1 — End-to-End Prototype (Weeks 1–2)

Objective: A working pipeline for Terraform files without RAG.

- Implement `ContextualAnalyzer` (heuristics + Checkov integration)
- Implement `FixGenerator` with base prompt (OpenAI GPT-4o-mini)
- Implement `ExternalToolValidator` (Checkov before/after comparison)
- Implement `PatchFormatter` (unified diff + CWE explanation)
- Wire `CentralAgent` for single-attempt mode
- Validate on `dataset/terraform/insecure_s3.tf`

Success criterion: The system detects at least one smell, generates a patch, and the patch is validated by Checkov (PVR > 0 on the Terraform subset).

7.3.2 Phase 2 — Knowledge Base and RAG (Weeks 3–4)

Objective: Integrate ChromaDB and measure RAG's contribution (RQ2).

- Create `dataset/taxonomy/smells_taxonomy.json` (62 entries from War et al. [7])
- Index taxonomy into ChromaDB with `all-MiniLM-L6-v2` embeddings
- Integrate `KnowledgeBase` and `KnowledgeRetriever`
- Evaluate Hit Rate@5 and Context Recall (RAGAS [8])
- Extend coverage to Ansible and Kubernetes

Success criterion: Hit Rate@5 ≥ 0.75; PVR improvement of at least +10 points over Phase 1 (Config B vs. Config C in the ablation study).

7.3.3 Phase 3 — CRAG Retry Loop (Week 5)

Objective: Activate the retry mechanism and measure its benefit (RQ3).

- Implement query refinement in `KnowledgeRetriever` (3 strategies)
- Enable retry loop in `CentralAgent` (max 3 attempts)
- Measure $\Delta R@2$ and $\Delta R@3$ on the full dataset

Success criterion: $\Delta R@3 > 0.10$ (retry loop adds >10 points).

7.3.4 Phase 4 — Full Evaluation (Week 6)

Objective: Produce the evaluation results for the thesis memoir.

- Implement `scripts/evaluate.py` (all 18 metrics, 5 runs, ablation)
- Compute Fleiss' Kappa (Eq. 5.11) across 5 runs
- Track API token usage and cost per script [5]
- Document failure cases (which smells are not corrected and why)

Chapter 8

Conclusion and Perspectives

8.1 Summary

This report has described the full design of an agentic framework for IaC security misconfiguration analysis and remediation. The framework transforms the passive RAG paradigm — a simple "answer provider" that cannot verify its own recommendations — into an active, critical agent that:

1. Detects security smells using both static analysis (Checkov) and heuristic contextual analysis.
2. Retrieves relevant knowledge from a vector database populated from the 62-category taxonomy of War et al. [7].
3. Generates candidate patches using an LLM with confidence-filtered output.
4. Validates each patch independently using Checkov [3], ensuring the fix is correct before it is returned to the developer.
5. Iterates up to three times with a refined RAG query (CRAG [11]) if no valid patch is found.

The evaluation dataset (27 smell instances, 8 files, 4 IaC tools), the 18-metric evaluation methodology, and the preliminary baseline results (Config A: Macro-F1 = 0.082 for Checkov alone) provide the empirical foundation to answer the five research questions in the next phase of the project. The baseline confirms the low ceiling of rule-based tools and quantifies the gap that the proposed framework must close.

8.2 Perspectives

Short term: Complete Phase 1 implementation (end-to-end Terraform prototype) and validate the core hypothesis that external tool validation improves patch correctness compared to LLM-only generation.

Medium term: Extend the dataset by mining GitHub repositories for real-world IaC files, following the large-scale evaluation approach of GLITCH [4] (196,755 scripts). Add a second annotator and measure inter-rater agreement with Cohen's Kappa [27].

Long term: Explore fine-tuning an LLM on the GenSIAC dataset [6] to specialise the Fix Generator for IaC security patches. Investigate the integration of GLITCH [4] as an additional validator for Ansible and Chef, providing deeper coverage than Checkov alone.

Bibliography

[1] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *Proceedings of the 41st IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE, 2019, pp. 164–175.

[2] A. War, A. Diallo, A. Habib, J. Klein, and T. F. Bissyande, "Vulnerabilities in infrastructure as code: What, how many, and who?" 2025. [Online]. Available: https://arxiv.org/abs/2505.01568

[3] Bridgecrew, "Checkov – static analysis for infrastructure as code," https://www.checkov.io, 2024.

[4] N. Saavedra and J. a. F. Ferreira, "GLITCH: Automated polyglot security smell detection in infrastructure as code," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE/ACM, 2022, pp. 1–12.

[5] G. De Vito, F. Palomba, and F. Ferrucci, "SecLLM: Enhancing security smell detection in IaC with large language models," *IEEE Access*, vol. 13, pp. 204 480–204 498, 2025.

[6] Y. Li, M. Grella, D. Nahmias, G. Engelberg, D. Klein, G. Guizzardi, T. van Ede, and A. Continella, "GenSIAC: Toward security-aware infrastructure-as-code generation with large language models," 2025. [Online]. Available: https://arxiv.org/abs/2505.01568

[7] A. War, S. L. B. Nikiema, J. Samhi, J. Klein, and T. F. Bissyande, "Security smells in infrastructure as code scripts: A taxonomy update beyond the seven sins," 2025. [Online]. Available: https://arxiv.org/abs/2511.12884

[8] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2024. [Online]. Available: https://arxiv.org/abs/2309.15217

[9] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "SELF-REFINE: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2303.17651

[10] A. War, A. N. Rawass, A. K. Kabore, J. Samhi, J. Klein, and T. F. Bissyande, "Detection of security smells in IaC scripts through semantics-aware code and language processing," 2025. [Online]. Available: https://arxiv.org/abs/2509.18790

[11] S.-Q. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, "CRAG: Comprehensive retrieval augmented generation," 2024. [Online]. Available: https://arxiv.org/abs/2401.15884

[12] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.11511

[13] H. Bezine, "Proposition d'un sujet de mastere MRSI: Framework agentique pour l'analyse et la remédiation des misconfigurations IaC," 2026, thesis subject proposal, date limite de dépôt: 28 janvier 2026.

[14] A. Rahman, M. R. Rahman, C. Parnin, and L. Williams, "Security smells in ansible and chef scripts: A replication study," *ACM Transactions on Software Engineering and Methodology*, vol. 30, no. 1, pp. 1–31, 2021.

[15] A. Rahman, E. Farhana, C. Parnin, and L. Williams, "Gang of eight: A defect taxonomy for infrastructure as code scripts," in *Proceedings of the 42nd IEEE/ACM International Conference on Software Engineering (ICSE)*, 2020, pp. 752–764.

[16] S. Dalla Palma, D. Di Nucci, F. Palomba, and D. A. Tamburri, "Within-project defect prediction of infrastructure-as-code using product and process metrics," *IEEE Transactions on Software Engineering*, vol. 48, no. 6, pp. 2086–2104, 2022.

[17] D. Gnieciak and T. Szandala, "Large language models versus static code analysis tools: A systematic benchmark for vulnerability detection," 2025. [Online]. Available: https://arxiv.org/abs/2506.18795

[18] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," 2023. [Online]. Available: https://arxiv.org/abs/2302.04761

[19] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, "Active retrieval augmented generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. [Online]. Available: https://arxiv.org/abs/2305.06983

[20] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang, "An empirical study of the non-determinism of ChatGPT in code generation," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 2, pp. 1–28, 2025.

[21] X. Lyu, S. Wang, C. Li, and C. Liu, "PatchEval: A benchmark for evaluating LLMs on patching real-world vulnerabilities," 2024. [Online]. Available: https://arxiv.org/abs/2511.11019

[22] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008. [Online]. Available: https://nlp.stanford.edu/IR-book/

[23] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[24] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[25] J. L. Fleiss, B. Levin, and M. C. Paik, *Statistical Methods for Rates and Proportions*, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.

[26] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.

[27] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[28] S. Reis, R. Abreu, M. d'Amorim, and D. Fortunato, "Leveraging practitioners' feedback to improve a security linter," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2022, pp. 1–12.